

PostgreSQL — Reference Card

psql · DDL · DML · SELECT · JOINS · CTE · window · JSON · UPSERT · indexes · transactions · plans. Stack: PostgreSQL ≥15 (MERGE, IDENTITY, generated cols) · SQL:2023 · Modern: `RETURNING`, `ON CONFLICT`, `LATERAL`, `FILTER`, materialised CTEs

◆ Query anatomy (order OF EXECUTION, not writing)

```
-- WRITE order          -- EXECUTION order (know both)
SELECT ...              -- 5. SELECT projections
FROM t                  -- 1. FROM (+ JOINS)
JOIN u ON ...           -- 2. ON / JOIN filters
WHERE ...               -- 3. WHERE
GROUP BY ...            -- 4. GROUP BY + aggregates
HAVING ...              -- 6. HAVING
ORDER BY ...            -- 7. ORDER BY
LIMIT n OFFSET m;       -- 8. LIMIT / OFFSET
```

**Alias order:** aliases from SELECT are NOT visible in WHERE/GROUP BY (they run before SELECT). But they ARE visible in ORDER BY (runs after). Use a CTE or subquery to sidestep.

◆ Starter · minimal CRUD skeleton

```
-- DDL (modern PG: IDENTITY, GENERATED, CHECK)
CREATE TABLE trades (
  id          bigint          GENERATED ALWAYS AS IDENTITY
  PRIMARY KEY,
  symbol      text            NOT NULL CHECK (symbol ~ '^[A-Z]+$'),
  qty         numeric(18,4) NOT NULL,
  price       numeric(18,4) NOT NULL,
  gross       numeric(18,4) GENERATED ALWAYS AS (qty * price)
  STORED,
  ts          timestamptz     NOT NULL DEFAULT now(),
  tags        jsonb           NOT NULL DEFAULT '{}'::jsonb
);
CREATE INDEX ON trades (symbol, ts DESC);           -- covering + sort
CREATE INDEX ON trades USING gin (tags);            -- JSONB search

-- DML (always use RETURNING; batch when possible)
INSERT INTO trades (symbol, qty, price)
VALUES ('AAPL', 100, 180.5), ('MSFT', 50, 310.0)
RETURNING id, gross;

-- SELECT
SELECT symbol, sum(gross) FILTER (WHERE qty > 0) AS long_gross
FROM trades WHERE ts >= now() - interval '7 days'
GROUP BY symbol HAVING sum(gross) > 1000
ORDER BY long_gross DESC LIMIT 10;

-- UPDATE + DELETE with RETURNING (audit)
UPDATE trades SET price = 181.0 WHERE id = 42 RETURNING *;
DELETE FROM trades WHERE ts < now() - interval '1 year'
RETURNING id;
```

● DDL — define schema (modern)

```
CREATE TABLE users (
  id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  -- (not SERIAL – legacy)
  email       citext NOT NULL UNIQUE,
  -- case-insensitive text
  created     timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT email_looks_ok CHECK (email ~ '@');

-- Generated columns (STORED = computed on write, indexable)
ALTER TABLE trades ADD COLUMN gross numeric GENERATED ALWAYS AS (qty*price) STORED;

-- Foreign keys with cascade
CREATE TABLE orders (
  id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  user_id     bigint NOT NULL REFERENCES users(id) ON DELETE CASCADE);

-- Enum types
CREATE TYPE status AS ENUM ('open', 'filled', 'cancelled');
```

```
-- Partitioning (large tables)
CREATE TABLE events (id bigint, dt timestampz, ...) PARTITION
BY RANGE (dt);
CREATE TABLE events_2026 PARTITION OF events
FOR VALUES FROM ('2026-01-01') TO ('2027-01-01');

-- Drop / alter
ALTER TABLE trades RENAME COLUMN qty TO quantity;
ALTER TABLE trades ADD COLUMN venue text;
DROP TABLE IF EXISTS temp_data CASCADE;

Use IDENTITY not SERIAL (SQL standard, safer around insert-explicit-id).
timestampz not timestamp (stores UTC internally).
```

● SELECT essentials

```
SELECT DISTINCT ON (symbol) symbol, price, ts          -- latest
FROM trades ORDER BY symbol, ts DESC;

SELECT * FROM trades
WHERE ts BETWEEN '2026-01-01' AND '2026-12-31'
  AND symbol IN ('AAPL', 'MSFT', 'GOOG')
  AND tags ->> 'strategy' = 'momentum'
  AND qty IS NOT NULL;

-- Boolean / conditional
SELECT symbol,
  CASE WHEN qty > 0 THEN 'long'
        WHEN qty < 0 THEN 'short'
        ELSE 'flat' END AS side,
  COALESCE(tags->>'venue', 'unknown') AS venue,
  NULLIF(gross, 0) AS gross_nz          -- returns NULL if
gross=0
FROM trades;

-- Existence / negation (prefer over IN for large sets)
SELECT * FROM trades t WHERE EXISTS (SELECT 1 FROM users u WHERE
u.id=t.user_id);
SELECT * FROM trades t WHERE NOT EXISTS (SELECT 1 FROM refunds r
WHERE r.tid=t.id);
```

▲ JOINS

| Join            | Behaviour                                   |
|-----------------|---------------------------------------------|
| INNER (default) | only matches                                |
| LEFT            | all left + matches (NULL for missing right) |
| RIGHT           | all right + matches                         |
| FULL OUTER      | all rows both sides                         |
| CROSS           | Cartesian product                           |
| LATERAL         | correlated: right side sees left row        |
| USING (col)     | shortcut for ON a.col=b.col + collapses col |

```
SELECT o.id, u.email, u.name
FROM orders o
JOIN users u ON u.id = o.user_id          -- users.id
+ orders.user_id (see DDL above)
LEFT JOIN discounts d ON d.user_id = u.id AND d.active
WHERE o.status = 'open';
-- USING (col) only works when BOTH tables have SAME-NAMED
column

-- LATERAL — top-N per group (elegant)
SELECT u.id, top3.*
FROM users u
CROSS JOIN LATERAL (
  SELECT o.id, o.amount
  FROM orders o WHERE o.user_id = u.id
  ORDER BY o.amount DESC LIMIT 3
) top3;

Cartesian trap: missing ON clause → CROSS JOIN silently. Turn on set
client_min_messages='warning'; to catch stealth cross-products.
```

▲ Aggregates + GROUP BY + FILTER

```
SELECT symbol,
  count(*)          AS n,
  count(*) FILTER (WHERE qty > 0) AS long_n,
  sum(gross)        AS total,
  sum(gross) FILTER (WHERE qty > 0) AS long_gross,
  avg(price),
  percentile_cont(0.5) WITHIN GROUP (ORDER BY price) AS
median,
  string_agg(DISTINCT venue, ' ' ORDER BY venue) AS
venues,
  array_agg(id ORDER BY ts) AS ids
FROM trades
WHERE ts >= now() - interval '30 days'
GROUP BY symbol
HAVING count(*) > 5
ORDER BY total DESC;

-- Subtotals: GROUPING SETS / ROLLUP / CUBE
SELECT region, product, sum(gross)
FROM sales
GROUP BY ROLLUP (region, product);    -- (region,product) +
(region) + total

FILTER (WHERE ...) replaces sum(CASE WHEN ...) — cleaner + slightly
faster.
```

◆ Window functions (aggregate without collapsing)

```
SELECT
  symbol, ts, price,
  -- Ranking
  row_number() OVER w          AS rn,    --
1,2,3 (no ties)
  rank() OVER w                AS rk,    --
1,2,2,4 (gaps)
  dense_rank() OVER w          AS drk,   --
1,2,2,3 (no gaps)
  percent_rank() OVER w        AS pct_rk,
  ntile(4) OVER w              AS quartile,

  -- Aggregate (over frame)
  sum(price) OVER w AS running_sum,
  avg(price) OVER (PARTITION BY symbol ORDER BY ts
ROWS BETWEEN 6 PRECEDING AND CURRENT
ROW) AS ma7,

  -- Neighbours
  lag(price,1) OVER w AS prev_price,
  lead(price,1) OVER w AS next_price,
  first_value(price) OVER w AS first_px,
  last_value(price) OVER (PARTITION BY symbol ORDER BY ts
ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED
FOLLOWING) AS last_px
FROM trades
WINDOW w AS (PARTITION BY symbol ORDER BY ts);
-- WINDOW named clause – DRY when reusing the same partition/
order many times
```

| Frame clause        | Meaning                                               |
|---------------------|-------------------------------------------------------|
| ROWS N PRECEDING    | fixed count of rows                                   |
| RANGE N PRECEDING   | values within N of current (needs numeric/date ORDER) |
| GROUPS N PRECEDING  | PG 11+; per ORDER BY tie group                        |
| UNBOUNDED PRECEDING | from start of partition                               |
| UNBOUNDED FOLLOWING | to end of partition                                   |

**Default frame** when using ORDER BY without explicit frame = RANGE  
UNBOUNDED PRECEDING (running total by value-of-ORDER-BY). Silently wrong for  
last\_value() — specify frame explicitly.

◆ CTE (WITH) + materialization hints

```
-- Simple CTE (inlined by optimizer in PG 12+ – single-
reference, non-recursive, no side-effects)
WITH recent AS (
    SELECT * FROM trades WHERE ts >= now() - interval '7 days'
)
SELECT symbol, sum(gross) FROM recent GROUP BY symbol;

-- Force materialization (optimizer barrier – used when CTE is
expensive OR affects semantics)
WITH heavy AS MATERIALIZED (
    SELECT * FROM slow_view
)
SELECT * FROM heavy WHERE ...;

WITH cheap AS NOT MATERIALIZED (
    even if referenced multiple times
    SELECT symbol FROM trades WHERE ts > now() - interval '1h'
) SELECT count(*) FROM cheap;

-- Recursive CTE (hierarchies, path enumeration)
WITH RECURSIVE ancestors AS (
    SELECT id, parent_id, 1 AS depth FROM tree WHERE id =
42
    -- base case
    UNION ALL
    SELECT t.id, t.parent_id, a.depth +
1
    -- step case
    FROM tree t JOIN ancestors a ON t.id = a.parent_id
) SELECT * FROM ancestors;

-- Data-modifying CTEs (multi-statement in one round-trip)
WITH deleted AS (DELETE FROM staging RETURNING *)
INSERT INTO archive SELECT * FROM deleted;
```

▲ UPSERT — ON CONFLICT vs MERGE

```
-- ON CONFLICT (idiomatic PG since 9.5 – simple, fast)
INSERT INTO trades (id, symbol, qty)
VALUES (1, 'AAPL', 100)
ON CONFLICT (id) DO UPDATE
    SET qty = EXCLUDED.qty,
    -- EXCLUDED = the row
    we tried to insert
    updated_at = now()
WHERE trades.qty IS DISTINCT FROM EXCLUDED.qty -- skip no-op
updates
RETURNING id;

-- MERGE (PG 15+ – SQL standard, better for complex cases)
MERGE INTO trades t
USING (VALUES (1, 'AAPL', 100), (2, 'MSFT', 50)) AS s(id,
symbol, qty)
ON t.id = s.id
WHEN MATCHED AND s.qty IS NULL THEN DELETE
    -- ← ORDER
matters: first matching branch wins
WHEN MATCHED AND t.qty IS DISTINCT FROM s.qty THEN
    UPDATE SET qty = s.qty, updated_at = now()
WHEN NOT MATCHED THEN
    INSERT (id, symbol, qty) VALUES (s.id, s.symbol, s.qty);
-- PG 17+: MERGE ... RETURNING supported.
-- Also common: INSERT ... ON CONFLICT (pk) DO NOTHING
(idempotent-insert)
```

| ON CONFLICT                            | MERGE                                         |
|----------------------------------------|-----------------------------------------------|
| simple upsert (1 outcome per conflict) | complex: match/insert/delete in one conflict) |
| requires a UNIQUE/PK constraint        | any join condition                            |
| fast + concurrent-safe                 | NOT concurrent-safe (needs SERIALIZABLE)      |

▲ JSON / JSONB

```
-- Extract
SELECT payload -> 'user'
    , payload ->> 'email'
    , payload #> '{user,addr,city}'
    , payload #>> '{user,addr,city}'
FROM events;
-- returns jsonb
-- returns text
-- deep path (jsonb)
-- deep path (text)
```

```
-- Filter (indexed via GIN)
SELECT * FROM events WHERE payload @> '{"user":
{"country":"US"}}'; -- contains
SELECT * FROM events WHERE payload ?
'session id'; -- has key
SELECT * FROM events WHERE payload ?| ARRAY('a',
'b'); -- has any key
```

```
-- SQL/JSON path (PG 12+ standard)
SELECT jsonb_path_query(payload, '$.items[*] ? (@.price > 100)')
FROM events;
```

```
-- Build + modify
INSERT INTO events (payload)
VALUES ('{"user": {"id": 42, "roles": ["admin"]}}':jsonb);
UPDATE events SET payload = payload || '{"processed":
true}':jsonb;
UPDATE events SET payload = jsonb_set(payload, '{user,name}',
"Alice");
```

```
-- Aggregate rows into JSON
SELECT jsonb_agg(row_to_json(t)) FROM trades t WHERE
symbol='AAPL';
```

▲ Arrays + Ranges

```
-- Arrays
SELECT ARRAY[1, 2, 3];
SELECT id FROM trades WHERE tags->'venues' @> '["NYSE"]':jsonb;
SELECT unnest(ARRAY[1,2,3]);
SELECT array_agg(id ORDER BY ts) FROM trades;
SELECT tags[1], tags[2:3] FROM t;
indexed slice
```

```
-- Ranges (typed intervals with EXCLUDE-based constraints)
CREATE TABLE bookings (
    room text, span tstzrange,
    EXCLUDE USING gist (room WITH =, span WITH &&) -- no
overlaps per room
);
INSERT INTO bookings VALUES ('A',
tstzrange('2026-07-01','2026-07-05','[]'));
-- overlap query
SELECT * FROM bookings WHERE span &&
tstzrange('2026-07-03','2026-07-04','[]');
```

● Indexes — types + production-safe DDL

| Type            | When                                                                                  |
|-----------------|---------------------------------------------------------------------------------------|
| btree (default) | =, <, >, BETWEEN, ORDER BY                                                            |
| hash            | equality-only; rarely worth it over btree (since PG 10 WAL-logged, no size advantage) |
| gin             | JSONB, arrays, full-text                                                              |
| gist            | range, geo, exclusion constraints                                                     |
| brin            | huge sorted tables (time-series); tiny footprint                                      |

```
-- Production-safe (never blocks writes; slower)
CREATE INDEX CONCURRENTLY idx_trades_symbol_ts
ON trades (symbol, ts DESC)
INCLUDE (gross)
WHERE ts >= '2026-01-01';
-- covering – avoid heap fetch
-- partial – smaller + faster
```

```
CREATE UNIQUE INDEX ON users (lower(email));
(case-insens login)
```

```
-- Rebuild without lock (PG 12+)
REINDEX INDEX CONCURRENTLY idx_trades_symbol_ts;
```

```
-- Check unused / bloat
SELECT indexname, idx_scan FROM pg_stat_user_indexes ORDER BY
idx_scan; -- 0 = unused
```

Never CREATE INDEX without CONCURRENTLY on a live table — blocks writes for the entire build.

● Transactions + isolation

| Level | Reads see |
|-------|-----------|
|-------|-----------|

|                          |                                              |
|--------------------------|----------------------------------------------|
| READ COMMITTED (default) | latest committed at each statement           |
| REPEATABLE READ          | snapshot at txn start (no ghosts within-txn) |
| SERIALIZABLE             | as if serial; retry on conflict              |

```
BEGIN ISOLATION LEVEL SERIALIZABLE;
    UPDATE accounts SET balance = balance - 100 WHERE id = 1;
    UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;
SerializationFailure → retry loop in app
```

```
-- Queue pattern (SKIP LOCKED – never blocks; PG 9.5+)
BEGIN;
    SELECT id, payload FROM jobs
    WHERE status = 'queued'
    ORDER BY created at
    LIMIT 1 FOR UPDATE SKIP LOCKED; -- claim one job, don't wait
for locked rows
    UPDATE jobs SET status='running' WHERE id = :id;
COMMIT;
```

```
-- Savepoints (partial rollback)
BEGIN;
    INSERT ...; SAVEPOINT sp1;
    UPDATE ...; ROLLBACK TO sp1; -- undo the UPDATE only
COMMIT;
```

◆ EXPLAIN — read a plan

```
EXPLAIN (ANALYZE, BUFFERS, VERBOSE, SETTINGS, WAL, FORMAT TEXT)
SELECT symbol, sum(gross) FROM trades
WHERE ts >= '2026-01-01'
GROUP BY symbol ORDER BY 2 DESC LIMIT 10;
-- SETTINGS (PG 12+): non-default GUCs that affected the plan
-- WAL (PG 13+): WAL bytes generated by the query (writes only)
```

| Symptom in plan               | Likely fix                                      |
|-------------------------------|-------------------------------------------------|
| Seq Scan on large table       | add index on filter col; or accept if >10% rows |
| Nested Loop, high rows        | rewrite as Hash Join (bigger work_mem)          |
| rows= WAY off actual          | stats stale → ANALYZE table                     |
| Sort spilled to disk          | increase work_mem or index the sort key         |
| Bitmap Heap Scan (many pages) | consider covering INCLUDE index                 |
| Filter after index            | index-only-scan blocked; add col to INCLUDE     |

◆ Dates / timestamps / intervals

```
SELECT now(), current_date, current_time, statement_timestamp();
```

```
-- Arithmetic
SELECT ts + interval '1 day', ts - interval '2 hours';
SELECT date_trunc('day', ts) AS day;
SELECT date_trunc('week', ts) AS week; -- Mon start
SELECT date_trunc('month', ts) AS month;
```

```
-- Extract parts
SELECT extract(year FROM ts), extract(dow FROM ts); -- 0=Sun
```

```
-- Time-zone handling
SELECT ts AT TIME ZONE 'UTC'; -- convert
timestampz → naive
SELECT ts AT TIME ZONE 'America/New_York';
```

```
-- Ranges (business-day style checks)
SELECT ts BETWEEN '2026-01-01' AND '2026-12-31';
SELECT extract(dow FROM ts) NOT IN (0, 6); -- weekday
```

```
-- Overlap
SELECT tstzrange(start, end) && tstzrange(now(), now()+interval
'1 day');
```

◆ Strings

```
SELECT length(s), lower(s), upper(s), trim(s), initcap(s);
SELECT substring(s, 1, 5), position('a' IN s), split_part(s,
',', 2);
SELECT concat_ws('-', 'a','b',NULL,'c'); -- 'a-b-c' (NULL-
safe join)
SELECT regexp_replace(s, '\s+', ' ', 'g'); -- collapse
```

```
whitespace
SELECT s ~* '^[a-z]+$';           -- ~* case-
insensitive regex
SELECT s LIKE 'foo%';             -- SQL wildcards %

SELECT similarity(a, b) FROM t;     -- pg_trgm – fuzzy
match

-- Full-text search
SELECT to_tsvector('english', 'Postgres is fast') @@
to_tsquery('fast');
-- Index for FTS:
CREATE INDEX ON docs USING gin (to_tsvector('english', body));
```

◆ Maintenance (the things that bite in prod)

- **VACUUM / autovacuum** — reclaims dead tuples. Watch bloat: `SELECT * FROM pg_stat_user_tables ORDER BY n_dead_tup DESC`;
- **ANALYZE** — refreshes stats for the planner. Runs with `VACUUM` (autovacuum default).
- **REINDEX CONCURRENTLY** — rebuilds bloated index without blocking.
- **pg\_stat\_activity** — see live queries: `SELECT pid, state, query FROM pg_stat_activity`;
- **pg\_stat\_statements** — top queries by total time (enable in `postgresql.conf`).
- **Long-running txn**: `SELECT pid, xact_start FROM pg_stat_activity WHERE xact_start < now()-interval '5 min'`;
- **Kill query**: `SELECT pg_cancel_backend(pid)`; (soft) or `pg_terminate_backend(pid)` (kill).
- **Backups**: `pg_dump -Fc db > db.dump` then `pg_restore -d newdb db.dump`.
- **Migrations**: Alembic / Flyway / sqitch. Never migrate schema without a tested rollback.

◆ psql meta-commands

| Cmd       | Purpose        |
|-----------|----------------|
| \l        | list databases |
| \c dbname | connect        |

|                      |                                                             |
|----------------------|-------------------------------------------------------------|
| \dt                  | list tables                                                 |
| \d table             | describe (cols, indexes, FKs)                               |
| \d+ table            | above + storage, comments                                   |
| \du                  | list roles                                                  |
| \df fn               | describe function                                           |
| \ef fn               | open function definition in editor                          |
| \timing on           | show query time                                             |
| \x                   | expanded (vertical) output — for wide rows                  |
| \copy t FROM 'f.csv' | client-side CSV load (unlike SQL COPY which is server-side) |
| \watch 2             | rerun last query every 2s                                   |
| \i file.sql          | execute file                                                |

▲ Top traps (6-month memory)

- **NULL ≠ NULL** — `WHERE x = NULL` matches nothing; use `x IS NULL`.
- **NULL + anything = NULL** — arithmetic silently returns `NULL`. Use `COALESCE(x, 0)`.
- **count(\*) vs count(col)** — `count(col)` skips `NULL`s; `count(*)` counts rows.
- **Chained aggregate not allowed** — `sum(avg(x))` raises. Wrap inner in a CTE.
- **DISTINCT ON** needs a matching leading `ORDER BY`. Missing = non-determinism.
- **SERIAL** is legacy — use `GENERATED ALWAYS AS IDENTITY`.
- **timestamp without tz** — silently drops `tz` info. Use `timestamptz`.
- **CREATE INDEX without CONCURRENTLY** blocks writes.
- **Function calls in WHERE** (`WHERE lower(email)=...`) skip indexes unless there's a functional index.
- **LIKE 'foo%'** uses `btree`; `LIKE '%foo'` or `'%foo%'` does **NOT** — use `pg_trgm GIN`.
- **OR in WHERE** often defeats indexes. Rewrite as `UNION ALL`.
- **Window frame default** is `RANGE UNBOUNDED PRECEDING` — silently wrong for `last_value`.
- **UPDATE without WHERE** — hits every row. Always test in a transaction first.

- **Long-running IDLE IN TRANSACTION** — blocks vacuum. Set `idle_in_transaction_session_timeout`.
- **MERGE not concurrent-safe** — needs `SERIALIZABLE` + `retry. ON CONFLICT` is.

◆ Quick-reference (recall in 5 seconds)

| Task                   | Canonical pattern                                                       |
|------------------------|-------------------------------------------------------------------------|
| Auto-increment PK      | <code>id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY</code>         |
| Timestamp              | <code>ts timestamptz NOT NULL DEFAULT now()</code>                      |
| Case-insensitive text  | <code>citext</code> extension                                           |
| Latest per group       | <code>DISTINCT ON (grp) ... ORDER BY grp, ts DESC</code>                |
| Top-N per group        | <code>LATERAL join with LIMIT</code>                                    |
| Conditional aggregate  | <code>sum(x) FILTER (WHERE cond)</code>                                 |
| Rolling 7 mean         | <code>window OVER (ORDER BY ts ROWS 6 PRECEDING)</code>                 |
| Rank in group          | <code>dense_rank() OVER (PARTITION BY g ORDER BY x)</code>              |
| Upsert                 | <code>INSERT ... ON CONFLICT (pk) DO UPDATE SET ... = EXCLUDED.x</code> |
| Queue claim            | <code>SELECT ... FOR UPDATE SKIP LOCKED LIMIT 1</code>                  |
| JSON extract text      | <code>payload -&gt;&gt; 'key'</code>                                    |
| JSON contains          | <code>payload @&gt; '{"k":"v"}' + GIN index</code>                      |
| Case-insensitive index | <code>CREATE UNIQUE INDEX ON t (lower(email));</code>                   |
| Prod-safe index        | <code>CREATE INDEX CONCURRENTLY ...</code>                              |
| Partial index          | <code>... WHERE ts &gt;= '2026-01-01'</code>                            |
| Covering index         | <code>... INCLUDE (col1, col2)</code>                                   |
| Recursive tree         | <code>WITH RECURSIVE t AS (base UNION ALL step)</code>                  |
| Batch insert           | <code>INSERT ... VALUES (r1),(r2),... RETURNING</code>                  |
| Serializable retry     | <code>catch SerializationFailure → sleep + retry</code>                 |
| Read a plan            | <code>EXPLAIN (ANALYZE, BUFFERS) &lt;query&gt;</code>                   |
| Kill query             | <code>SELECT pg_cancel_backend(pid);</code>                             |
| Load CSV (client)      | <code>\copy t FROM 'f.csv' CSV HEADER</code>                            |